RAG Systems

Interview Preparation Guide

Chunking Strategies & Caching Architecture

55+ Scenario-Based, Concept & System Design Questions

For AI Engineer & AI Product Manager Roles

Based on: Chunking Strategies — The Definitive Guide & RAG Caching System

February 2026

How to Use This Guide

This document contains 55+ non-basic interview questions organized by type: scenario-based, architecture, concept deep-dives, product/PM-focused, evaluation, advanced edge cases, and operations. Each question includes key points that should be addressed in your answer — not full answers, but the critical elements an interviewer would expect to hear.

Section	Focus Area	Questions	Best For
A	Chunking Scenarios	8	AI Engineer
B	Caching Scenarios	6	AI Engineer
C	Architecture & System Design	5	Both Roles
D	Concept Deep-Dives	6	Both Roles
E	Product & Decision-Making	5	AI PM
F	Evaluation & QA	3	Both Roles
G	Advanced & Edge Cases	7	Senior AI Engineer
H	Tricky Conceptual & Comparisons	8	Senior AI Engineer
I	Integration & Operations	3	Both Roles

SECTION A: Scenario-Based Questions — Chunking Strategies

Q1. You are building a RAG system for a law firm that needs to process 50,000 legal contracts. Each contract has clauses referencing other clauses, tables of terms, and defined terms sections. Walk through your chunking strategy.

Key Points to Include in Your Answer:

- Start with document-structure-aware chunking (detect sections, clauses, schedules)
- Handle cross-references with contextual chunking (Anthropic's approach) to prepend clause context
- Use parent-child indexing: small child chunks for precise clause retrieval, parent chunks for full section context
- Special table handling for term definition tables (keep whole or row-level with headers)
- Metadata enrichment: clause number, contract ID, party names, effective date
- Proposition-based chunking for key defined terms and obligations
- Evaluation: create 50+ QA pairs from real contracts, measure Recall@K and MRR

Q2. Your team deployed a RAG system with recursive character splitting at 512 tokens. Users report that questions about financial data return wrong numbers. Diagnose and fix.

Key Points to Include in Your Answer:

- Identify the 'Table Massacre' failure mode: tables split across chunks losing header-data relationships
- Add table detection as a pre-processing step before chunking
- Implement multi-representation strategy: raw table as one chunk + LLM-generated natural language summary as another
- Consider row-level chunking with headers prepended for large tables
- Add content_type metadata to distinguish text vs table chunks
- Use tools like unstructured.io or docling for PDF table extraction
- Validate fix by running financial-specific QA evaluation set

Q3. You're processing a 200-page research paper with heavy cross-referencing ('as discussed in Section 3.2', 'see Figure 5'). Standard chunking loses these references. How do you handle this?

Key Points to Include in Your Answer:

- This is the 'Orphan Reference' failure mode
- Apply contextual chunking: LLM prepends document context to each chunk before embedding
- Use prompt caching to amortize the cost (99 of 100 calls use cached document prefix)
- Parent-child hierarchy: section-level parents with paragraph-level children
- Metadata: section_hierarchy field (e.g., 'Ch 3 > Methods > Data Collection')
- Consider late chunking if using a long-context embedding model to preserve transformer attention across references
- Tradeoff discussion: contextual chunking costs more but handles cross-references better than late chunking for precision-critical use cases

Q4. Your enterprise RAG system handles PDFs, Markdown docs, Slack exports, and Python code files. Design the ingestion pipeline's chunking layer.

Key Points to Include in Your Answer:

- Build a type-detection router that identifies document type and dispatches to appropriate strategy

- PDF: structure-aware splitting + table detection + contextual chunking
- Markdown: MarkdownHeaderTextSplitter by headings, then recursive splitting within large sections
- Slack exports: conversational chunking by topic/thread, keep Q&A pairs together
- Python code: AST-based chunking via tree-sitter, prepend class/module context and imports
- Hybrid approach: structure-aware first pass, recursive/semantic splitting within oversized sections
- Unified metadata schema across all types: source_document, content_type, document_date, entities

Q5. A product manager asks you to implement semantic chunking because it 'sounds smarter.' How do you evaluate whether it's worth switching from recursive splitting?

Key Points to Include in Your Answer:

- Reference the Oct 2024 research paper questioning semantic chunking's cost-effectiveness
- Calculate cost: semantic chunking embeds every individual sentence before producing final chunks (100K sentences = 100K API calls just for chunking)
- Design A/B test: same embedding model, same LLM, same top-K, only change chunking strategy
- Create evaluation set of 50–100 QA pairs from actual user queries
- Measure retrieval metrics (Recall@K, Precision@K, MRR, nDCG@10) and end-to-end metrics (faithfulness, correctness)
- Discuss the 'start simple, upgrade with evidence' decision framework
- Consider document characteristics: semantic chunking helps more for topic-diverse documents than for homogeneous content

Q6. You're building a customer support bot. 60% of queries are near-duplicates. Your top-5 retrieval returns 3 chunks that are 80% identical. What's happening and how do you fix it?

Key Points to Include in Your Answer:

- Identify the 'Duplicate Flood' failure mode caused by excessive sliding window overlap
- Reduce overlap percentage (from 30%+ to 10–15%)
- Add post-retrieval deduplication step
- Implement a reranker that naturally penalizes near-duplicates (e.g., Cohere Rerank, cross-encoder)
- Consider switching from overlap-based to sentence-level chunking for cleaner boundaries
- For FAQ-style content, use document structure chunking: each Q&A pair = one chunk
- Evaluate with Precision@K to measure ratio of unique, relevant chunks in top-K

Q7. Your CEO wants the RAG system to handle both quick factoid questions ('What year was X founded?') and complex analytical questions ('Compare Q3 and Q4 revenue trends'). One chunk size won't work. What do you do?

Key Points to Include in Your Answer:

- Implement parent-child (hierarchical) chunking as the core pattern
- Small children (128–256 tokens) indexed for factoid precision, large parents (1024–2048 tokens) returned for context
- Use LlamaIndex's HierarchicalNodeParser + AutoMergingRetriever for auto-merging when multiple children from same parent are retrieved
- Multi-level hierarchy: document > section > paragraph > sentence
- Query routing: classify query type and adjust retrieval strategy (top-K, parent expansion)
- Chunk size quick reference: small for factoid, medium for explanatory, large for synthesis/comparison
- Evaluate both query types separately in your evaluation set

Q8. You need to chunk a codebase of 10,000 Python files for a code search RAG system. Walk through your approach.

Key Points to Include in Your Answer:

- Use AST-based chunking via tree-sitter for accurate structure detection
- Natural boundaries: functions, classes, methods as individual chunks
- Prepend context: class name, module path, relevant imports with each chunk
- Pair each function chunk with its docstring for documentation retrieval
- Handle long functions: sub-split at logical blocks if exceeding chunk size limit
- Metadata: file_path, class_name, function_name, dependencies, language
- Consider code-specific embedding models (e.g., CodeBERT, StarCoder embeddings) vs general-purpose models

SECTION B: Scenario-Based Questions — RAG Caching System

Q9. Your RAG system serves 10,000 queries/day. The PM says latency is 5 seconds average and wants it under 500ms. Design a caching strategy.

Key Points to Include in Your Answer:

- Propose 3-tier cache: Exact (hash match) → Semantic (embedding similarity ≥0.95) → Retrieval (chunk cache ≥0.90)
- Tier 1 gives 1–10ms response at $0 cost, Tier 2 gives 50–200ms at ~$0.00002, Tier 3 saves retrieval cost but still calls LLM
- With ~60% query redundancy, most queries hit Tier 1 or 2
- Cache promotion: Tier 3 hits auto-populate Tier 1+2 for future speed
- Invalidation via TTL (7 days exact/semantic, 1 day retrieval) + doc_version bumps on new uploads
- Project cost savings: $360/mo → $148/mo at 60% hit rate
- Backend choice: SQLite for dev/single server, Redis for production/multi-server

Q10. After deploying caching, users report stale answers — the knowledge base was updated 2 days ago but cached answers still reflect old data. Diagnose and propose solutions.

Key Points to Include in Your Answer:

- Root cause: cache entries created before the document update still pass TTL check (7-day TTL)
- Current fix: bump_doc_version() on document upload invalidates ALL entries globally
- Limitation: global invalidation is too aggressive — unrelated entries also invalidated
- Propose per-document invalidation: track which doc chunks contributed to each cached answer, only invalidate affected entries
- Add semantic re-validation: periodically re-check if cached answers still hold given new documents
- Manual fallback: POST /cache/clear endpoint for full wipe
- Discuss tradeoff: aggressive invalidation = fresh answers but lower hit rate; lenient = higher hit rate but risk of stale data

Q11. Your Tier 2 semantic cache is getting slow. Investigation shows 50,000 cached embeddings and cosine similarity scan takes 200ms. How do you scale?

Key Points to Include in Your Answer:

- Current approach: brute-force scan loads ALL embeddings and computes cosine similarity in Python
- At 50K entries: ~50ms; at 100K+: unacceptable latency
- Solution: migrate to Approximate Nearest Neighbor (ANN) algorithms
- Options: FAISS (local, Facebook), HNSW graphs (sub-ms at 1M+), RediSearch (native in Redis), Qdrant (managed vector DB)
- If using Redis backend, leverage RediSearch module for native vector similarity
- Add LRU eviction to cap cache size and prevent unbounded growth
- Consider TTL reduction to naturally limit active entry count

Q12. Two users ask semantically similar questions but expect different answers because they have different permission levels (one is an admin, one is a regular user). Your semantic cache returns the admin's answer to the regular user. Fix this.

Key Points to Include in Your Answer:

- The cache key must incorporate user context/role, not just query content

- Modify the hash (Tier 1) and embedding lookup (Tier 2/3) to include role/permission as part of the cache key
- Option 1: namespace caches by user role (separate cache partitions)
- Option 2: composite key = hash(normalize(query) + role)
- For Tier 2/3, filter semantic matches by matching role metadata before returning
- Broader principle: any context that affects the answer must be part of the cache key (tenant, role, language, etc.)
- Test: same question from different roles should produce cache misses against each other

Q13. You're migrating from SQLite to Redis for production. What are the key architectural considerations and risks?

Key Points to Include in Your Answer:

- Strategy Pattern ensures zero code changes in api.py — just change CACHE_BACKEND env var
- Redis advantages: native TTL (SETEX), atomic INCR for doc_version, sub-ms reads, thousands of concurrent clients
- SQLite limitations: WAL mode supports ~10 concurrent connections, manual TTL cleanup in Python
- Risk: Redis TTL race condition — key expires but SET index still contains UUID; need index cleanup pattern (GET returns nil → SREM)
- Data migration: may need to warm the cache or accept cold start
- Infrastructure: Docker container, consider managed services (AWS ElastiCache, Redis Cloud, Upstash) for production
- Monitoring: use RedisInsight for key browsing, TTL inspection, memory usage tracking

Q14. The cosine similarity threshold for Tier 2 is set at 0.95. Users complain that obvious rephrasings like 'revenue of Apple' vs 'Apple's revenue' aren't hitting the cache. What do you do?

Key Points to Include in Your Answer:

- Lower the threshold (e.g., to 0.92 or 0.90) and measure impact on false positive rate
- Risk of lowering: semantically different questions get matched (false positives → wrong answers)
- Improve query normalization: more aggressive preprocessing (synonym replacement, stop word removal)
- Consider using a better embedding model (e.g., upgrade from text-embedding-3-small to text-embedding-3-large)
- Add a lightweight reranker or cross-encoder as a verification step after cosine match
- Implement threshold tuning pipeline: collect query pairs, manually label same/different intent, find optimal threshold
- A/B test different thresholds and monitor both hit rate AND answer correctness

SECTION C: Architecture & System Design Questions

Q15. Design a production RAG pipeline from document ingestion to response generation. Focus on how chunking and caching interact.

Key Points to Include in Your Answer:

- Ingestion: type detection → structure extraction → chunking (strategy per type) → metadata enrichment → embedding → vector DB
- Query time: normalize → Tier 1 exact cache → embed → Tier 2 semantic cache → Tier 3 retrieval cache → full pipeline
- Chunking feeds the quality of what gets cached; bad chunks = cached bad answers
- Cache invalidation triggers on document re-ingestion (bump version)
- Parent-child indexing: cache stores parent chunk IDs for context expansion
- Hybrid search: BM25 + dense retrieval, with caching applied before both
- Monitoring: retrieval metrics, cache hit rates, latency percentiles, cost tracking

Q16. Your company has 5 product teams each building their own RAG system. Design a shared chunking and caching platform.

Key Points to Include in Your Answer:

- Abstract chunking as a service with configurable strategies per team/use-case
- Central config: each team registers their document types and preferred strategies
- Shared cache with tenant isolation (namespaced keys: team:tier:hash)
- Common metadata schema enforced across all teams
- Evaluation framework as a shared service: teams submit QA pairs, platform runs A/B tests
- Swappable backends: teams can choose SQLite (dev) or Redis (prod) independently
- Cost attribution: track per-team embedding, LLM, and cache storage costs

Q17. You need to serve RAG responses with <200ms P99 latency for a real-time customer-facing application. How do you architect this?

Key Points to Include in Your Answer:

- Cache is the primary path — target 80%+ hit rate for Tier 1/2
- Pre-warm cache with anticipated queries from search logs/analytics
- Redis backend (not SQLite) for sub-ms reads and high concurrency
- ANN index (FAISS/HNSW) for Tier 2/3 instead of brute-force scan
- Async pipeline: return cached answer immediately, trigger background refresh if near TTL expiry
- Edge caching: CDN-level caching for most common queries
- Fallback UX: if cache misses, show ‘generating answer...’ with streaming response rather than blocking

Q18. How would you implement contextual chunking (Anthropic’s approach) at scale for 1 million documents while keeping costs manageable?

Key Points to Include in Your Answer:

- Use prompt caching: process full document once, subsequent chunk contextualization calls use cached prefix
- For 100-chunk document: 100 LLM calls, but 99 benefit from cached prefix (dramatically cheaper)
- Batch processing: parallelize contextual enrichment across documents

- Selective application: apply contextual chunking only to high-value document types (financial, legal), use recursive splitting for simpler docs
- Cost estimation: calculate per-document cost and total corpus cost before committing
- Quality validation: contextual chunking achieved 35–67% retrieval failure reduction in Anthropic's benchmarks
- Compare against late chunking (Jina) for cost-sensitive portions: zero additional LLM calls, uses transformer attention instead

Q19. Design a cache warming strategy for a RAG system that will be deployed to a new region with zero historical data.

Key Points to Include in Your Answer:

- Analyze query logs from existing regions to identify top 1000 most frequent queries
- Pre-compute answers for these queries and populate Tier 1 (exact) and Tier 2 (semantic) caches
- Generate synthetic rephrasings of top queries using an LLM to populate semantic cache breadth
- Pre-compute and cache retrieval results (Tier 3) for common document sections
- Staged rollout: route 10% traffic initially, let cache build organically, then scale up
- Monitor cold-start metrics: cache miss rate, average latency, cost per query during warm-up period
- Set aggressive TTLs initially (shorter), then extend as cache stabilizes

SECTION D: Concept-Deep-Dive Questions

Q20. Explain the tradeoff between late chunking (Jina) and contextual retrieval (Anthropic). When would you choose one over the other?

Key Points to Include in Your Answer:

- Late chunking: embed full document first, then split token embeddings → each chunk carries transformer attention context from entire doc
- Contextual retrieval: chunk first, then prepend LLM-generated context to each chunk before embedding
- Cost: late chunking has zero additional LLM calls; contextual requires one LLM call per chunk
- Model requirement: late chunking needs long-context embedding model (e.g., Jina v2/v3 8K tokens) with mean pooling
- Precision: contextual retrieval gives explicit, human-readable context; late chunking's context is implicit in embeddings
- Scale: late chunking better for large corpora (cost-sensitive); contextual better for high-value, precision-critical docs
- Document length: late chunking limited by embedding model's context window; contextual works with any embedding model

Q21. Why is parent-child chunking considered the most important pattern in production RAG? What problems does it solve that other strategies don't?

Key Points to Include in Your Answer:

- Solves the fundamental small-vs-large chunk dilemma by decoupling retrieval unit from context unit
- Small chunks = precise embeddings (less dilution) but lack context; large chunks = rich context but noisy retrieval
- Parent-child gives you both: small children indexed for retrieval precision, large parents returned for LLM context
- Implementation is minimal: store parent_id with each child chunk
- Auto-merging variant: if enough children from same parent match, automatically merge to parent level
- Works with any base chunking strategy (recursive, semantic, structure-aware)
- Consistently significant quality improvement across use cases with minimal overhead

Q22. Explain the Chunk Quality Principle and how it connects to the embedding model's effectiveness.

Key Points to Include in Your Answer:

- Golden Rule: a good chunk should be independently meaningful when read in isolation
- Embedding models encode the semantic meaning of the entire chunk into a single vector
- If the chunk is incoherent (broken sentences, mixed topics), the embedding becomes diluted and noisy
- A 'Frankenchunk' (end of one topic + start of another) produces an embedding that matches neither topic well
- Token-based counting aligns with how embedding models actually process text (they have token limits, not character limits)
- The most underrated evaluation: manually read 20 random chunks and ask if they make sense in isolation
- This principle cascades: bad chunks → bad embeddings → bad retrieval → bad LLM answers → bad user experience

Q23. What is the role of cosine similarity thresholds in a semantic cache, and how do you tune them?

Key Points to Include in Your Answer:

- Cosine similarity measures angle between two embedding vectors: 1.0 = identical meaning, 0.0 = unrelated
- Tier 2 uses 0.95 threshold (strict): catches rephrasings while avoiding false matches
- Tier 3 uses 0.90 threshold (loose): catches same-context queries that need same chunks
- Too high threshold: low hit rate, many cache misses on valid rephrasings
- Too low threshold: high false positive rate, wrong cached answers returned
- Tuning method: collect labeled query pairs (same intent vs different intent), compute similarity, find threshold that maximizes correct matches
- Threshold is model-dependent: different embedding models produce different similarity distributions for the same query pairs

Q24. Discuss the Strategy Pattern as used in the cache backend design. Why is it superior to a monolithic implementation?

Key Points to Include in Your Answer:

- Abstract base class (CacheBackend) defines WHAT the cache must do (11 methods), not HOW
- SQLite and Redis are two independent implementations of the same interface
- Factory function (get_cache_backend()) creates the right backend based on env var
- Singleton pattern prevents creating new DB connections per request
- api.py never knows or cares which backend is active → zero code changes when switching
- Extensibility: adding a new backend (e.g., DynamoDB, Memcached) only requires implementing the interface
- Testability: can mock the interface for unit tests without needing a real database

Q25. Why does the caching system store embeddings as BLOBs (binary) instead of JSON arrays? What's the engineering tradeoff?

Key Points to Include in Your Answer:

- Binary (numpy float32 bytes): 6KB per 1536-dim vector
- JSON array: ~50KB per 1536-dim vector (8x larger)
- Storage savings directly impact SQLite file size and Redis memory usage
- Faster I/O: less data to read/write from disk or network
- Deserialization cost: binary → numpy array is faster than JSON parse → list → numpy conversion
- Tradeoff: binary is not human-readable for debugging; JSON is inspectable
- For production systems handling thousands of embeddings, the 8x storage savings is significant

SECTION E: Product & Decision-Making Questions (AI PM Focus)

Q26. The VP of Engineering asks: 'Should we build or buy our chunking pipeline?' How do you frame this decision?

Key Points to Include in Your Answer:

- Buy (LangChain/LlamaIndex): fast time-to-market, community-tested, but less control over edge cases
- Build: full control over document-type routing, metadata schema, evaluation, but higher engineering cost
- Hybrid recommended: use LangChain splitters as building blocks, build custom routing and evaluation layers
- Key factors: number of document types, quality requirements, team expertise, iteration speed needs
- Consider: unstructured.io for parsing, custom pipeline for chunking strategy selection and metadata
- Cost analysis: engineering time to build vs. ongoing cost of working around library limitations
- Decision framework: start with LangChain defaults, measure, build custom only where measurements show gaps

Q27. Your RAG system's retrieval quality is poor. The ML engineer wants to upgrade the embedding model; the data engineer wants to fix chunking. You have budget for one. Which do you prioritize?

Key Points to Include in Your Answer:

- Chunking first: the guide states most RAG quality issues originate in the chunks, not the embedding model
- Garbage in, garbage out: a better embedding model encoding broken chunks still produces broken retrieval
- Diagnostic approach: manually inspect 20 random chunks — if they're incoherent, fix chunking first
- Check for common failure modes: cliffhangers, frankenchunks, orphan references, table massacres
- Embedding model upgrade gives marginal improvement if the input (chunks) is already high quality
- Cost comparison: fixing chunking is usually a configuration/pipeline change; new embedding model may require re-indexing everything
- Evidence-based: run the evaluation framework on current chunks with current model vs improved chunks with current model

Q28. A customer asks for real-time RAG responses (sub-second). Currently your pipeline takes 5 seconds. How do you roadmap this?

Key Points to Include in Your Answer:

- Phase 1 (Quick wins): implement 3-tier caching → covers 60% of queries at <200ms
- Phase 2 (Optimization): add BM25 hybrid search, reranking, query routing
- Phase 3 (Infrastructure): Redis backend, ANN index for semantic cache, pre-warming
- Phase 4 (Advanced): streaming responses, edge caching, smart query router (ML-based cache skip prediction)
- Metrics to track at each phase: P50/P95/P99 latency, cache hit rate, cost per query, answer quality
- UX consideration: perceived latency matters — streaming the LLM response as it generates feels faster even at same total time

- Tradeoff transparency: sub-second for all queries may require sacrificing freshness (longer TTLs) or answer quality (returning less-precise cached results)

Q29. How would you measure and report ROI of implementing caching in your RAG system to executive stakeholders?

Key Points to Include in Your Answer:

- Cost metrics: API spend before vs after (LLM calls, embedding calls, vector DB queries)
- Projected savings: at 10K queries/day and 60% hit rate, $212/month saved (per the caching guide's analysis)
- Latency metrics: average response time reduction (5000ms → 5ms for exact hits, 125ms for semantic hits)
- User experience: reduction in user drop-off due to slow responses
- Scalability: 10x more users at same API cost due to cache absorbing traffic spikes
- Operational: reduced load on vector DB and LLM APIs during peak hours
- Dashboard: Grafana with real-time hit rates, latency percentiles, and cumulative cost savings

Q30. Your RAG system needs to comply with data residency requirements (EU data stays in EU). How does this affect your caching and chunking architecture?

Key Points to Include in Your Answer:

- Cache backend must be region-local: Redis instances deployed per region, not shared globally
- Embedding API calls must route to region-compliant endpoints (EU OpenAI endpoint or self-hosted models)
- Document chunks and their metadata must not cross regional boundaries
- Cache invalidation must be region-scoped (doc_version per region)
- Contextual chunking LLM calls must use region-compliant models
- Audit trail: log which documents contributed to which cached answers for compliance audits
- Consider self-hosted embedding models to avoid sending data to third-party APIs

SECTION F: Evaluation & Quality Assurance Questions

Q31. Design a comprehensive evaluation framework to compare 3 different chunking strategies for your production RAG system.

Key Points to Include in Your Answer:

- Create 50–100 QA pairs from real documents; for each, note the source section containing the answer
- Control variables: same embedding model, same LLM, same top-K, only change chunking strategy
- Retrieval metrics: Recall@K (did we find right chunks?), Precision@K (did we avoid irrelevant ones?), MRR (how quickly?), nDCG@10 (ranking quality)
- End-to-end metrics: Faithfulness (grounded in retrieved chunks?), Answer Relevancy, Answer Correctness
- Tools: RAGAS, DeepEval, or custom evaluation scripts
- Manual inspection: print 20 random chunks per strategy and assess coherence
- Statistical significance: run multiple trials, report confidence intervals, not just averages

Q32. How would you detect and handle chunk quality degradation over time as new document types are added to the corpus?

Key Points to Include in Your Answer:

- Continuous monitoring: track retrieval metrics (Recall@K, MRR) over time with a fixed evaluation set
- Alerting: set thresholds on metric degradation (e.g., Recall@K drops >5% triggers alert)
- New document type detection: classify incoming documents and flag when unseen types appear
- Periodic chunk inspection: automated sampling of chunks + LLM-based quality assessment
- A/B testing: when adding new document types, test chunking strategy impact before full rollout
- Failure mode monitoring: check for each common failure mode (cliffhanger, frankenchunk, etc.) programmatically
- Feedback loop: track user thumbs-down or ‘answer not helpful’ signals back to specific chunks

Q33. Your cache hit rate is 40% but you expected 60%. How do you diagnose and improve this?

Key Points to Include in Your Answer:

- Analyze query distribution: are queries truly repetitive or mostly unique?
- Check Tier 1 misses: is normalization too strict? (e.g., not handling typos, different punctuation)
- Check Tier 2 misses: is the 0.95 threshold too high? Lower to 0.92 and measure false positive rate
- Check TTL: are entries expiring too quickly? Analyze time between repeated queries
- Check doc_version: are frequent document uploads invalidating the entire cache too aggressively?
- Query clustering analysis: embed all queries, cluster, identify clusters with no cache entries
- Pre-warming: identify high-frequency queries from logs and pre-populate cache

SECTION G: Advanced & Edge-Case Questions

Q34. How do you handle chunking for a multilingual document corpus (English, Chinese, Arabic) where token counts and sentence boundaries work differently?

Key Points to Include in Your Answer:

- Token-based counting varies dramatically: Chinese characters often = 1 token each, while English words average 1.3 tokens
- Sentence segmentation: spaCy/NLTK models are language-specific; need per-language NLP pipeline
- Character-based languages (Chinese, Japanese) may need different chunk size thresholds
- Arabic/Hebrew RTL text needs special handling in structure detection
- Embedding model selection: multilingual models (e.g., Cohere multilingual) vs language-specific models
- Cache considerations: same question in different languages should be different cache entries unless cross-lingual matching is desired
- Test chunking quality per language separately with language-specific evaluation sets

Q35. Your RAG system needs to handle versioned documents (e.g., v1, v2, v3 of the same policy document). How do you manage chunks and cache across versions?

Key Points to Include in Your Answer:

- Include document version in chunk metadata for filtering at retrieval time
- Default behavior: retrieve from latest version unless user specifies otherwise
- Cache key must include document version context to avoid serving stale answers
- Incremental update strategy: only re-chunk and re-embed changed sections, not entire document
- Diff-aware chunking: detect what changed between versions for targeted re-indexing
- Per-document cache invalidation (improvement over current global invalidation)
- Consider maintaining a version timeline for temporal queries ('What changed in the policy between v1 and v3?')

Q36. An engineer proposes using LLM-based (agentic) chunking for the entire 500K document corpus. Evaluate this proposal.

Key Points to Include in Your Answer:

- Cost analysis: agentic chunking requires multiple LLM calls per document → thousands of dollars for 500K docs
- Latency: ingestion pipeline would be orders of magnitude slower
- Quality: highest possible chunk quality — LLM understands meaning, resolves references, detects topic shifts
- Recommendation: NOT justified for entire corpus; reserve for high-value subset (top 5–10% most critical docs)
- Hybrid approach: agentic for legal/financial docs, recursive splitting for general content
- Cost-quality tradeoff table from the guide: agentic = Very High cost, Highest quality
- Alternative: use contextual chunking (cheaper) for most docs, agentic only where contextual falls short

Q37. How do you handle the 'cold start' problem when deploying a new RAG system with caching? The first N users get slow responses.

Key Points to Include in Your Answer:

- Pre-warm strategy: analyze expected query patterns from product requirements or user research
- Synthetic query generation: use LLM to generate likely questions for each document section
- Staged rollout: internal beta users first to warm the cache before public launch
- Tier 3 promotion: even if Tier 1/2 miss initially, Tier 3 hits automatically promote to faster tiers
- Self-learning cache: system gets smarter over time as cache promotion fills Tier 1+2
- Set user expectations: show loading indicators, use streaming responses during cold start period
- Monitor: track cache hit rate over first 7 days, expect logarithmic improvement as cache warms

Q38. You're building a RAG system for medical documents where hallucination could be dangerous. How do you design chunking and caching to maximize faithfulness?

Key Points to Include in Your Answer:

- Proposition-based chunking for key medical facts: decompose into atomic, verifiable statements
- Contextual chunking to preserve critical context (drug interactions, dosage conditions, contraindications)
- Small chunks (128–256 tokens) for precise retrieval of specific medical facts
- Strict metadata: source document, page, section, publication date, evidence level
- Caching: lower TTLs for medical content (guidelines change frequently), aggressive version invalidation
- Evaluation: faithfulness metric must be primary — every answer must be grounded in retrieved chunks
- Human-in-the-loop: flag low-confidence answers for medical professional review before caching

Q39. Your Tier 3 retrieval cache caches chunks, but two different questions that need the same chunks might need very different answers. How do you prevent incorrect answer reuse?

Key Points to Include in Your Answer:

- By design, Tier 3 only caches the retrieved chunks, NOT the answer — it still calls the LLM for generation
- The 0.90 threshold means only queries needing similar context share cached chunks
- Risk: if two questions need the same chunks but different reasoning, the LLM gets the same context and generates appropriately
- Cache promotion: the new answer (from LLM + cached chunks) is promoted to Tier 1+2 for that specific question
- If false matches occur: raise the Tier 3 threshold (e.g., to 0.93) to reduce context-sharing between dissimilar questions
- Monitor: track cases where Tier 3 hits produce low-quality answers and adjust threshold accordingly
- Key insight: Tier 3 saves retrieval cost, not LLM cost — the answer is always freshly generated

Q40. Design a chunking evaluation that doesn't require ground truth labels (unsupervised evaluation).

Key Points to Include in Your Answer:

- Chunk coherence score: embed each chunk, measure internal consistency (low variance = coherent)
- Boundary quality: check if chunk boundaries fall at natural topic transitions (low cross-boundary similarity = good split)
- Readability: can an LLM summarize the chunk meaningfully? If it says 'this chunk is incoherent', flag it
- Size distribution analysis: check for outliers (very small or very large chunks suggest problems)

- Overlap analysis: measure redundancy between adjacent chunks
- Self-contained test: prompt an LLM with just the chunk and ask ‘What topic does this cover?’ — if it can’t answer, the chunk fails the golden rule
- Comparison: run unsupervised metrics across multiple strategies to rank them before investing in labeled evaluation

SECTION H: Tricky Conceptual & Comparison Questions

Q41. Recursive character splitting is called 'the strongest baseline.' Why does it outperform fixed-size chunking so dramatically despite being purely syntactic?

Key Points to Include in Your Answer:

- It respects natural document structure (paragraphs, sentences, words) in priority order without NLP tools
- Graceful degradation: only breaks finer boundaries when absolutely necessary
- Fixed-size creates broken sentences and mixed-topic chunks; recursive preserves coherent units
- Combined with overlap, it avoids the 'Cliffhanger' failure mode
- It's deterministic, fast, and predictable — easy to debug in production
- The 512-token, 50-token overlap configuration balances precision and context for most use cases
- Many production systems never need to exceed this baseline — upgrade only with evidence from evaluation

Q42. Compare BM25 hybrid search with semantic-only search in the context of chunking quality. How does chunk quality affect each differently?

Key Points to Include in Your Answer:

- BM25 is keyword-based: works well even with imperfect chunks if key terms are present
- Semantic search relies on embedding quality: broken/incoherent chunks produce poor embeddings
- Hybrid (BM25 + semantic) is more robust to chunking imperfections than semantic-only
- Contextual chunking improves semantic search more than BM25 (BM25 doesn't use embeddings)
- Table chunks: BM25 can match exact numbers; semantic search struggles with numerical data in embedding space
- Anthropic's results: Contextual Embeddings + BM25 gave 49% retrieval failure reduction vs 35% for embeddings alone
- Recommendation: always use hybrid search as it compensates for chunking weaknesses

Q43. What happens when you use a high overlap percentage (30%+) with a reranker in the pipeline? Walk through the interaction effects.

Key Points to Include in Your Answer:

- High overlap creates near-duplicate chunks that waste the top-K retrieval budget
- Without reranking: top-5 might return 3 chunks that are 80% identical (the 'Duplicate Flood' failure)
- A cross-encoder reranker can naturally penalize near-duplicates by re-scoring based on query relevance AND diversity
- But reranking adds latency (~300ms) which partially offsets the benefit of overlap
- Better approach: use 10% overlap + reranker rather than 30% overlap + deduplication
- For caching: high-overlap chunks create more cache entries with near-identical embeddings, increasing Tier 2/3 scan time
- Net recommendation: overlap should be the minimum needed to prevent boundary information loss, not a safety blanket

Q44. Explain why sentence-level chunking doesn't solve the coreference problem and how Sentence Window Retrieval partially addresses it.

Key Points to Include in Your Answer:

- Coreference: ‘This is critically important’ has no meaning without the preceding sentence explaining what ‘this’ refers to
- Sentence-level chunking groups N sentences but doesn’t resolve pronouns or references across groups
- Sentence Window Retrieval: retrieve 3 sentences, but return those + N sentences before and after to the LLM
- This gives precision from small retrieval units (clean embeddings) with context from the expanded window
- LlamaIndex implements this as SentenceWindowNodeParser
- It’s essentially a lightweight version of parent-child chunking applied at the sentence level
- Limitation: the window size is fixed, so it may still miss long-range dependencies (references 20 sentences back)

Q45. The cache promotion pattern (Tier 3 → Tier 1+2) is described as making the cache ‘smarter over time.’ Analyze the conditions under which this self-learning breaks down.

Key Points to Include in Your Answer:

- Works well when query distribution is stable (same questions keep recurring)
- Breaks down with high query diversity: if every query is unique, Tier 3 hits are rare and promotions don’t compound
- Breaks down with frequent document updates: doc_version bumps invalidate all promoted entries
- Edge case: Tier 3 returns slightly wrong chunks → LLM generates slightly wrong answer → promoted to Tier 1+2 → wrong answer served 1000x faster
- Cold start problem: promotion requires at least 2 occurrences of similar queries to show benefit
- Global invalidation is the biggest enemy of promotion — per-document invalidation would preserve unaffected promotions
- Monitoring: track how many promoted entries actually get subsequent hits vs expire unused

Q46. You have a document with 50 pages of narrative text followed by 20 pages of dense data tables. A single chunking strategy won’t work optimally. Design a within-document routing approach.

Key Points to Include in Your Answer:

- Content-type detection as pre-processing: classify each page/section as narrative, tabular, code, or mixed
- Narrative sections: recursive character splitting (512 tokens, 50 overlap) or semantic chunking
- Table sections: keep-whole strategy for small tables, row-level with headers for large tables, LLM summary as additional representation
- Mixed sections: extract tables separately, chunk surrounding text independently
- Use content_type metadata on each chunk for retrieval filtering
- Multi-representation: some sections get both a raw chunk and an LLM-summarized chunk
- Evaluate each content type separately in your QA evaluation set

Q47. How does prompt caching (as used in contextual chunking) differ from the 3-tier RAG cache? When would you use both together?

Key Points to Include in Your Answer:

- Prompt caching: LLM API feature that caches the document prefix across multiple calls during indexing — reduces cost of contextual enrichment

- 3-tier RAG cache: query-time cache that stores results to avoid re-running the retrieval + generation pipeline
- They operate at different stages: prompt caching at indexing time, RAG cache at query time
- Use both: prompt caching reduces the cost of building high-quality contextual chunks; RAG caching reduces the cost of serving those chunks
- Prompt caching is ephemeral (per API session); RAG cache is persistent (days/weeks)
- Cost optimization: prompt caching makes contextual chunking affordable for large corpora; RAG caching makes serving affordable at scale
- Neither replaces the other — they address different cost bottlenecks in the pipeline

Q48. A junior engineer claims that increasing top-K from 5 to 20 will solve all retrieval quality problems. Evaluate this claim from a chunking and caching perspective.

Key Points to Include in Your Answer:

- More chunks ≠ better answers: LLM context window gets noisier with irrelevant chunks ('Lost Needle' at scale)
- Root cause matters: if chunks are poorly formed, retrieving more bad chunks makes answers worse
- Cost impact: 4x more chunks means more tokens in the LLM prompt = higher generation cost
- Caching impact: Tier 3 retrieval cache stores more chunks per entry, increasing storage and scan time
- Better solutions: improve chunk quality, add reranking to re-sort the top-K, use parent-child to get context without noise
- Precision@K will drop as K increases — more irrelevant chunks dilute the useful ones
- The right approach: optimize Recall@5 through better chunking, not Recall@20 through brute force

SECTION I: Integration & Operations Questions

Q49. How would you implement an A/B testing framework to compare two different chunking strategies in production without disrupting users?

Key Points to Include in Your Answer:

- Shadow mode: run both strategies in parallel, serve results from Strategy A, log Strategy B results for comparison
- Traffic splitting: route 10% of traffic to Strategy B, monitor quality metrics before scaling up
- Consistent assignment: same user always gets same strategy (hash user_id to strategy) to avoid confusion
- Shared evaluation: same QA evaluation set run against both strategies nightly
- Metrics: retrieval quality, end-to-end answer quality, latency, cost per query
- Rollback plan: feature flag to instantly revert to Strategy A if metrics degrade
- Duration: run for at least 2 weeks to capture query variety and achieve statistical significance

Q50. Your production RAG system processes 100 new documents per day. Design the incremental indexing pipeline that handles chunking, embedding, caching, and invalidation.

Key Points to Include in Your Answer:

- Document ingestion trigger: file upload, API webhook, or scheduled batch job
- Idempotent processing: hash document content to detect true changes vs re-uploads
- Incremental re-chunking: only process new/modified documents, not entire corpus
- Selective cache invalidation: bump version for affected documents only (not global)
- Embedding queue: batch new chunks for efficient embedding API calls
- Vector DB upsert: add new chunks, remove/update changed chunks
- Post-ingestion validation: run evaluation subset to confirm quality didn't degrade

Q51. How do you handle chunking and caching for a multi-tenant RAG platform where each tenant has different documents and different requirements?

Key Points to Include in Your Answer:

- Tenant isolation: each tenant's chunks and cache entries are namespaced (tenant_id prefix on all keys)
- Configurable strategies: each tenant can configure their preferred chunking strategy, chunk size, overlap
- Shared infrastructure: same Redis cluster, same embedding model, but logically separated data
- Per-tenant cache: doc_version tracked per tenant, not globally
- Resource limits: per-tenant quotas on cache size, embedding API calls, storage
- Cross-tenant optimization: if tenants share common document types, offer pre-configured templates
- Billing: track per-tenant API costs, cache storage, and compute for accurate billing

Interview Tips — Quick Reference

For AI Engineers

- Always start with the simplest approach and explain why you'd upgrade (the 'start simple, upgrade with evidence' framework)
- Reference specific failure modes by name (Cliffhanger, Frankenchunk, Orphan Reference, Table Massacre, Duplicate Flood, Lost Needle)
- Quantify tradeoffs: mention token counts, latency numbers, cost per query, storage sizes
- Show you understand evaluation: mention specific metrics (Recall@K, MRR, nDCG@10, Faithfulness)
- Demonstrate production thinking: metadata, monitoring, incremental updates, rollback plans

For AI Product Managers

- Frame technical decisions in terms of business impact: cost savings, latency improvement, user satisfaction
- Show you can prioritize: 'fix chunking before upgrading embedding model' with clear reasoning
- Understand the cost-quality-latency triangle and how caching shifts the tradeoff curve
- Reference the decision framework: diagnose failure modes, apply targeted fixes, measure impact
- Demonstrate cross-functional thinking: how engineering decisions affect UX, compliance, and scalability